



Understanding Interfaces in C#: A Comprehensive Guide

By Albert Harmon

Published August 4, 2017

🕒 4 min read

Education



TLDR This blog post provides a detailed explanation of interfaces in C#, highlighting their definition, differences from classes, implementation requirements, and practical examples. It serves as a foundational resource for understanding how interfaces

function within the C# programming language.

In the world of C# programming, interfaces play a crucial role in defining contracts for classes. This blog post aims to provide a thorough understanding of interfaces, their characteristics, and how they differ from classes. We will explore the definition of interfaces, their implementation, and practical examples to illustrate their use.

What is an Interface?

An interface in C# is a user-defined type that defines a contract for classes. It is similar to a class but has distinct characteristics. While a class can contain methods with bodies (non-abstract methods), an interface can only contain abstract methods, which do not have bodies. This fundamental difference sets the stage for how interfaces are utilized in C# programming.

Key Differences Between Classes and Interfaces

- **Class:** A class can contain both non-abstract methods (methods with bodies) and abstract methods (methods without bodies). It serves as a blueprint for creating objects and can encapsulate data and behavior.
- **Abstract Class:** An abstract class can contain both non-abstract and abstract methods. It cannot be instantiated directly and is meant to be inherited by other classes.

- **Interface:** An interface contains only abstract methods. All methods in an interface are implicitly public and abstract, meaning they do not require the `public` or `abstract` modifiers when declared.

The Purpose of Abstract Methods

Abstract methods in interfaces serve as a contract that must be fulfilled by any class that implements the interface. When a class inherits from an interface, it is required to provide implementations for all the abstract methods defined in that interface. This ensures that the class adheres to the contract specified by the interface.

Inheritance and Interfaces

In C#, a class can inherit from both another class and an interface simultaneously. This allows for a flexible design where a class can consume properties from a parent class while also implementing the methods defined in an interface.

Implementing an Interface

When a class implements an interface, it must provide concrete implementations for all the abstract methods declared in the interface. This is mandatory and cannot be skipped. The relationship can be visualized as follows:

- **Interface:** Defines abstract methods.
- **Child Class:** Implements the abstract methods of the interface.

Example of Interface Implementation

To illustrate how to define and implement an interface, consider the following steps:

1. **Define an Interface:** Create an interface using the `interface` keyword. For example:

```
public interface ITestInterface
{
    void Add(int a, int b);
    void Subtract(int a, int b);
}
```

2. **Implement the Interface:** Create a class that implements the interface and provides the method bodies:

```
public class ImplementationClass : ITestInterface
{
    public void Add(int a, int b)
    {
        Console.WriteLine(a + b);
    }
}
```

```
        public void Subtract(int a, int b)
        {
            Console.WriteLine(a - b);
        }
    }
```

3. **Using the Implementation:** Create an instance of the implementing class and call the methods:

```
ImplementationClass obj = new ImplementationClass();
obj.Add(5, 3); // Outputs: 8
obj.Subtract(5, 3); // Outputs: 2
```

Important Rules for Interfaces

- **Default Scope:** The default access modifier for members of an interface is public. In contrast, members of a class are private by default.
- **No Fields:** Interfaces cannot contain fields or variables. They can only declare methods.
- **Inheritance:** An interface can inherit from another interface, allowing for a hierarchical structure of interfaces.

Conclusion

Interfaces are a powerful feature in C# that promote a clean and organized approach to programming. They enforce a contract for classes, ensuring that certain methods are implemented, which enhances code reliability and maintainability. Understanding how to define and implement interfaces is essential for any C# developer, as it lays the groundwork for effective object-oriented programming.

For further learning, consider exploring additional resources on abstract classes and methods, as well as practical applications of interfaces in real-world scenarios.